

Praxis: Kernel konfigurieren und bauen

Einen Linux-Kernel nachvollziehbar konfigurieren, kompilieren und prüfen

1.09

Den Build als sichere Laborübung vorbereiten

LABORUMGEBUNG

VM oder Testsystem

Keine Produktionsmaschine;
bekannten funktionierenden Kernel und
Boot-Eintrag behalten.

RESSOURCEN

Platz und Arbeitsspeicher

Freien Speicher prüfen; Build-Größe
und Laufzeit hängen stark von
Konfiguration und Debug-Optionen ab.

BERECHTIGUNGEN

Ohne root bauen

Quellen und Build als normaler
Benutzer; sudo erst für ausdrücklich
geplante Installation.

Secure Boot, Signierung und Wiederherstellungsweg vor jeder Aktivierung ausdrücklich klären.

Build-Abhängigkeiten distributionsabhängig prüfen

BEISPIEL · DEBIAN / UBUNTU

```
$ sudo apt update
$ sudo apt install \
    bc binutils bison flex gcc git make \
    libelf-dev libssl-dev libncurses-dev \
    dwarves pahole openssl perl-base

$ gcc --version
$ make --version
```

01 Compiler und Kbuild

gcc, make, Binutils sowie Parser- und Build-Werkzeuge.

02 Optionale Features

ncurses für menuconfig; ELF, OpenSSL und pahole abhängig von Konfiguration und Distribution.

Paketnamen und Zusatzwerkzeuge richten sich nach Distribution, Architektur und Kernel-Features.

Quellcode holen und Build-Output getrennt halten

BASH · GLEICHE TERMINALSESSION

```
$ git clone --depth 1 \  
    https://git.kernel.org/pub/scm/linux/\   
    kernel/git/torvalds/linux.git linux-src  
$ cd linux-src  
$ build_dir=/tmp/kernel-build  
$ mkdir -p "$build_dir"  
$ git rev-parse --short HEAD
```

01 Commit festhalten

Exakten Quellstand dokumentieren, damit der Build später reproduzierbar bleibt.

02 Output separieren

O= verweist bei allen weiteren make-Aufrufen auf dasselbe Build-Verzeichnis.

Ab jetzt arbeiten alle Beispiele aus dem Source Tree und verwenden dieselbe build_dir-Variable.

Eine Ausgangskonfiguration übernehmen und sichern

NUR EINE DER BEIDEN VARIANTEN VERWENDEN

```
$ cp "/boot/config-$(uname -r)" "$build_dir/.config"
$ cp "$build_dir/.config" "$build_dir/.config.before"
$ make O="$build_dir" olddefconfig

# Alternative ohne bestehende Konfiguration:
$ make O="$build_dir" defconfig
```

01 Config übernehmen

Vorhandene Optionen übernehmen; neue Optionen erhalten Default-Werte.

02 Pfad ist optional

/boot/config-* existiert nicht auf jeder Distribution; dann defconfig oder geeignetes Config-Backup nutzen.

Die Sicherung der ursprünglichen .config macht spätere Änderungen sichtbar und überprüfbar.

Konfigurationsziele bewusst auswählen

defconfig

Architektur-Defaults als neuer Ausgangspunkt

olddefconfig

Bestehende Optionen behalten; neue auf Default setzen

menuconfig

Interaktive ncurses-Oberfläche mit Hilfetexten

localmodconfig

Reduziert anhand aktuell geladener Module

Vorsicht: localmodconfig kann Treiber für gerade nicht angeschlossene Hardware entfernen.

Kernel-Optionen mit menuconfig gezielt ändern

INTERAKTIVE KCONFIG-OBERFLÄCHE

```
$ make O="$build_dir" menuconfig

# Nach relevanten Symbolen suchen:
CONFIG_LOCALVERSION
CONFIG_IKCONFIG
CONFIG_IKCONFIG_PROC

# Hilfe und Abhängigkeiten prüfen,
# Änderungen speichern und beenden.
```

01 Eigenen Namen setzen

LOCALVERSION, zum Beispiel -training, unterscheidet den gebauten Kernel vom vorhandenen.

02 Config nachvollziehen

IKCONFIG und – falls verfügbar – IKCONFIG_PROC machen die aktive Konfiguration später leichter prüfbar.

Abhängigkeiten und Hilfetexte prüfen: Nicht jedes Symbol ist in jeder Konfiguration auswählbar.

Änderungen skriptfähig setzen und vergleichen

REPRODUZIERBARE KONFIGURATION

```
$ ./scripts/config \
  --file "$build_dir/.config" \
  --set-str LOCALVERSION "-training" \
  --enable IKCONFIG \
  --enable IKCONFIG_PROC
$ make O="$build_dir" olddefconfig
$ ./scripts/diffconfig \
  "$build_dir/.config.before" \
  "$build_dir/.config"
```

01 Einstellungen festhalten

Ein kurzer Konfigurationsschritt lässt sich im Labor oder in CI wiederholen.

02 Optionen auflösen

olddefconfig normalisiert den Stand; diffconfig zeigt tatsächliche Änderungen.

Der Config-Diff dokumentiert fachliche Änderungen klarer als ein vollständiger .config-Dump.

Die Bedeutung von y, m und n sicher lesen

CONFIG_FEATURE=y

Fester Bestandteil des Kernel-Images

CONFIG_FEATURE=m

Separat ladbares Kernelmodul

CONFIG_FEATURE is not set

Option deaktiviert: entspricht n

CONFIG_LOCALVERSION="-training"

String-Option statt tristate

Nur tristate-Optionen erlauben m; boolesche Optionen kennen ausschließlich y oder n.

Den Kernel parallel bauen und Fehler protokollieren

BASH · BUILD OHNE ROOT-RECHTE

```
$ jobs=$(nproc)
$ set -o pipefail
$ make O="$build_dir" -j"$jobs" 2>&1 | tee "$build_dir/build.log"
$ make O="$build_dir" kernelrelease
```

01 Parallelität anpassen

nproc ist ein Startwert; bei knappem RAM die Anzahl gleichzeitiger Jobs reduzieren.

02 Build-Fehler behalten

pipefail verhindert, dass ein erfolgreiches tee einen fehlgeschlagenen make-Befehl verdeckt.

Root-Rechte sind für die Kompilierung nicht erforderlich; der Build bleibt im Output-Verzeichnis.

Artefakte, Release und Kernelmodule überprüfen

ARTEFAKTE · X86-BEISPIEL

```
$ ls -lh "$build_dir/vmlinux"
$ ls -lh \
    "$build_dir/arch/x86/boot/bzImage"
$ find "$build_dir" -name '*.ko' | head
$ make O="$build_dir" kernelrelease
$ grep '^CONFIG_LOCALVERSION=' \
    "$build_dir/.config"
```

01 Image und ELF

vmlinux ist das gelinkte Kernel-ELF; der x86-Bootpfad verwendet typischerweise bzImage.

02 Module und Release

.ko-Dateien existieren nur für entsprechend konfigurierte Module; Release und Suffix prüfen.

Image-Pfad und erzeugte Zusatzartefakte hängen von Architektur, Konfiguration und Build-Ziel ab.

Optional: ARM64 per Cross-Compiler erzeugen

EMBEDDED · SEPARATER OUTPUT-TREE

```
$ arm_build=/tmp/kernel-build-arm64
$ make ARCH=arm64 \
    CROSS_COMPILE=aarch64-linux-gnu- \
    O="$arm_build" defconfig
$ make ARCH=arm64 \
    CROSS_COMPILE=aarch64-linux-gnu- \
    O="$arm_build" -j "$(nproc)" \
    Image dtbs modules
```

01 Toolchain prüfen

aarch64-linux-gnu-gcc
und passende Binutils
müssen verfügbar sein.

02 Zielartefakte

ARM64-Image, Device
Trees und konfigurierte
Module getrennt vom
x86-Build erzeugen.

ARCH, CROSS_COMPILE und O= müssen bei allen zusammengehörigen Aufrufen konsistent bleiben.

Typische Build-Probleme systematisch eingrenzen

DIAGNOSE · ABHÄNGIG VOM FEHLERBILD

```
$ make O="$build_dir" V=1  
$ make O="$build_dir" listnewconfig  
$ tail -n 40 "$build_dir/build.log"  
$ df -h "$build_dir"
```

FEHLENDE TOOLS

ELF · OpenSSL · pahole

Konfigurationsabhängige Pakete und exakte Fehlermeldungen prüfen.

KNAPPE RESSOURCEN

Speicher · -j reduzieren

Parallelität und freien Platz an die Laborumgebung anpassen.

SIGNIERUNG

Zertifikate klären

Distributionsschlüssel und Secure-Boot-Vertrauen ausdrücklich prüfen.

Optional installieren – nur nach Freigabe

OPTIONAL · ÄNDERT DAS TESTSYSTEM

```
$ make O="$build_dir" kernelrelease  
$ sudo make O="$build_dir" modules_install  
$ sudo make O="$build_dir" install
```

01 Boot-Integration prüfen

initramfs, Bootloader-Eintrag, Installations-Hooks und resultierenden Release vorab prüfen.

02 Fallback erhalten

Vorhandenen funktionierenden Kernel behalten;
Secure-Boot-Signaturen und Vertrauenskette klären.

Kein automatischer Neustart: Erst wenn Bootfähigkeit, Rückfalloption und Signierung geprüft sind.

Ergebnis dokumentieren und bewusst validieren

01 Quellstand und Zielarchitektur sichern

Git-Commit, Architektur, Toolchain-Version und eindeutigen Release dokumentieren.

02 Konfiguration und Änderung nachvollziehen

.config, gesicherte Ausgangskonfiguration und scripts/diffconfig-Ausgabe ablegen.

03 Build und Artefakte überprüfen

build.log, vmlinux, architekturspezifisches Image und konfigurierte Module kontrollieren.

04 Aktivierung separat entscheiden

Erst nach bewusstem Bootwechsel: uname -r, Fallback und gegebenenfalls /proc/config.gz prüfen.

Ein erfolgreicher Kernel-Build ist auch ohne Installation, Bootloader-Änderung oder Neustart wertvoll.